



深度学习

DEEP LEARNING-XIAOLIN HU

Part.02

数学和机器学习基础

● 大纲

- 数学基础
- 机器学习基础

● 数学基础

线性代数

● 数学对象

- 标量 (Scalar)

单个数字，通常用不加粗的小写字母表示，例如, a, b, x

- 向量 (Vector)

一维数字数组，通常用加粗的小写字母表示，例如, $\mathbf{a}, \mathbf{b}, \mathbf{x}$

- 矩阵 (Matrix)

二维数字数组，通常用加粗的大写字母表示，例如, $\mathbf{A}, \mathbf{B}, \mathbf{X}$

- 张量 (Tensor)

n维数字数组，通常用以下方式表示，例如, $\mathbf{A}, \mathbf{B}, \mathbf{X}$

$$\mathbf{A} = \left(\left(\begin{array}{cc} A_{1,1,1} & A_{1,2,1} \\ A_{2,1,1} & A_{2,2,1} \end{array} \right), \left(\begin{array}{cc} A_{1,1,2} & A_{1,2,2} \\ A_{2,1,2} & A_{2,2,2} \end{array} \right) \right)$$

有时可能会不遵守这些约定

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

$$\mathbf{A} = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}$$

● 简单运算

- 矩阵转置: $\mathbf{A}^\top$

$$\mathbf{A} = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \quad \mathbf{A}^\top = \begin{pmatrix} A_{1,1} & A_{2,1} \\ A_{1,2} & A_{2,2} \end{pmatrix}$$

- 向量可以视为特殊矩阵

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \quad \mathbf{a}^\top = (a_1, a_2, a_3)$$

- 矩阵乘法: 如果有 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 和 $\mathbf{B} \in \mathbb{R}^{n \times p}$, 则 $\mathbf{C} = \mathbf{AB}$ 的维度是 $m \times p$, 并且 $C_{i,j} = \sum_k A_{i,k} B_{k,j}$
- 元素积 (阿达玛乘积): $\mathbf{C} = \mathbf{A} \odot \mathbf{B}$ 其中三个矩阵具有相同的维度, 并且 $C_{i,j} = A_{i,j} B_{i,j}$

● 简单运算

- 性质

- $A(B + C) = AB + AC$
- $A(BC) = (AB)C$
- $AB \neq BA$ 但有 $x^T y = y^T x$
- $(AB)^T = B^T A^T$

- 单位矩阵: $I \in \mathbb{R}^{n \times n}$

- 对于 $x \in \mathbb{R}^n$, 有 $Ix = x$
- 对于 $A \in \mathbb{R}^{n \times m}$, 有 $IA = A$

- 方阵 A 的逆矩阵用 A^{-1} 表示, 使得 $A^{-1}A = I$

$$I = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix}$$

● 线性方程

• 线性方程组

$$\mathbf{Ax} = \mathbf{b}$$

其中 $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$ 且 $\mathbf{x} \in \mathbb{R}^n$, 也可以表示为

$$A_{1,1}x_1 + A_{1,2}x_2 + \cdots + A_{1,n}x_n = b_1$$

$$A_{2,1}x_1 + A_{2,2}x_2 + \cdots + A_{2,n}x_n = b_2$$

...

$$A_{m,1}x_1 + A_{m,2}x_2 + \cdots + A_{m,n}x_n = b_m$$

- 如果 $m = n$ 并且 $\mathbf{A}^{-1}$ 存在, 则上述等式存在唯一解 $\mathbf{x}^*$
- 如果 $m < n$, 则存在无数多个解
- 如果 $m > n$, 则可能无解!

欠定
超定

范数

- 范数用于衡量向量或矩阵的“大小”
- 向量的 L_p 范数定义为

$$||\mathbf{x}||_p = \left(\sum_i |x_i|^p \right)^{1/p}$$

其中 $p \in \mathbb{R}, p > 1$

- L_2 范数被称为 欧几里得范数, $||\mathbf{x}||_2 = \sqrt{\mathbf{x}^\top \mathbf{x}}$
- L_1 范数简化为 $||\mathbf{x}||_1 = \sum_i |x_i|$
- L_∞ 范数简化为 $||\mathbf{x}||_\infty = \max_i |x_i|$

● 范数

- 范数是满足下列条件的任意函数 f
 - $f(\mathbf{x}) \geq 0$
 - $f(\mathbf{x}) = 0 \Rightarrow \mathbf{x} = 0$
 - $f(\mathbf{x} + \mathbf{y}) \leq f(\mathbf{x}) + f(\mathbf{y})$ (三角不等式)
 - $f(\alpha \mathbf{x}) = |\alpha|f(\mathbf{x}), \forall \alpha \in \mathbb{R}$
- 矩阵的 Frobenius 范数

$$\|A\|_F = \sqrt{\sum_{i,j} A_{i,j}^2}$$

● 数学基础

概率论

● 随机变量

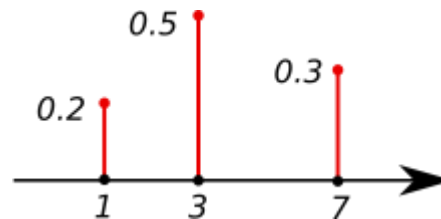
- **随机变量** 是可以随机取不同值的变量
 - 用 x 表示随机变量, 用 x_1 和 x_2 表示其两个可能的值
 - 对于向量, 我们将随机变量写为 $\mathbf{x}$, 其中一个值写为 x
 - 离散 与 连续



● 概率分布

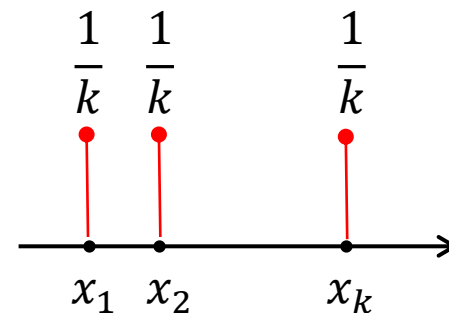
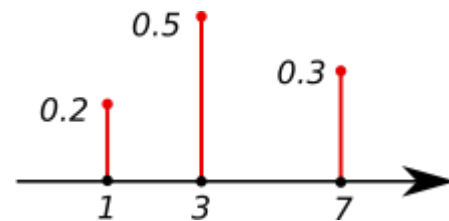
概率分布是一个随机变量或一组随机变量在每个可能状态下的可能性分布

- 离散变量的概率分布可以用概率质量函数 (PMF) 来表示
 - $x = x$ 的概率表示为 $P(x)$ 或 $P(x = x)$
 - $x \sim P(x)$ 指定了 x 服从的分布
- 联合概率分布
 - $P(x = x, y = y)$ 或 $P(x, y)$ 表示 $x = x$ 和 $y = y$ 同时出现的概率



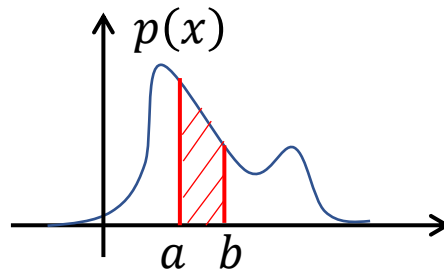
● 概率质量函数

- 作为随机变量 x 的PMF, 函数 P 需要满足:
 - P 的定义域必须是 x 的所有可能状态的集合
 - $\forall x \in x, 0 \leq P(x) \leq 1$
 - $\sum_{x \in x} P(x) = 1$
- 均匀分布
 - 考虑一个离散的随机变量 x 有 k 个不同的状态
 - $P(x = x_i) = \frac{1}{k}, \forall i$



● 概率密度函数

- 连续变量的概率分布可以用概率密度函数 (PDF) 来表示
- 作为PDF, 函数 p 需要满足下列性质
 - P 的定义域必须是 x 的所有可能状态的集合
 - $\forall x \in x, p(x) \geq 0$
 - $\int p(x)dx = 1$



是否可以有
 $p(x) > 1$?

- x 在区间 $[a, b]$ 中的概率为 $\int_a^b p(x)dx$
- 注意 $p(x)$ 不能直接代表特定状态的概率

● 边缘概率

假设我们知道一组变量的概率分布，其中一部分变量的概率分布称为**边缘概率分布**

- 设 $P(x, y)$ 表示离散随机变量 x 和 y 的概率分布，则有

$$P(x = x) = \sum_y P(x = x, y = y)$$

- 设 $p(x, y)$ 表示连续随机变量 x 和 y 的PDF，则有

$$p(x) = \int P(x, y) dy$$

● 条件概率

条件概率是在假定发生了某些事件的条件下，某个事件发生的概率

- 给定 $x = x$, $y = y$ 的条件概率表示为 $P(y = y|x = x)$, 计算方式为

$$P(y = y|x = x) = P(y = y, x = x)/P(x = x)$$

- 链式法则

$$P(x^{(1)}, \dots, x^{(n)}) = P(x^{(1)}) \prod_{i=2}^n P(x^{(i)} | x^{(1)}, \dots, x^{(i-1)})$$

- 练习:** 根据上述定义, 下式是否正确?

$$P(a, b, c) = P(a|b, c)P(b|c)P(c)$$

● 独立和条件独立

两个随机变量 x 和 y 是**独立**的，如果它们的联合概率分布可以表示为两个因子的乘积，其中一个因子只涉及 x ，另一个因子只涉及 y ：

$$\forall x \in \mathcal{X}, y \in \mathcal{Y},$$
$$p(x = x, y = y) = p(x = x)p(y = y)$$

表示为 $x \perp y$

给定随机变量 z ，两个随机变量 x 和 y 是**条件独立**的，如果 x 和 y 的联合条件概率分布可以按以下方式分解：

$$\forall x \in \mathcal{X}, y \in \mathcal{Y}, z \in \mathcal{Z},$$
$$p(x = x, y = y | z = z) = p(x = x | z = z)p(y = y | z = z)$$

表示为 $x \perp y | z$

● 期望

函数 $f(x)$ 的期望或期望值定义为，当 x 采样自概率分布 $P(x)$ 时，函数 f 的平均值

- 对于离散变量

$$\mathbb{E}_{x \sim P}[f(x)] = \sum_x P(x)f(x)$$

- 对于连续变量

$$\mathbb{E}_{x \sim P}[f(x)] = \int p(x)f(x)dx$$

- 如果概率分布 P 是已知的，也可以写作 $\mathbb{E}_x[f(x)]$
- 期望是线性的: 如果 α 和 β 不依赖于 x , 则有

$$\mathbb{E}_x[\alpha f(x) + \beta g(x)] = \alpha \mathbb{E}_x[f(x)] + \beta \mathbb{E}_x[g(x)]$$

● 方差与协方差

- 函数 $f(x)$ 的方差

$$\text{Var}(f(x)) = \mathbb{E}[(f(x) - \mathbb{E}[f(x)])^2]$$

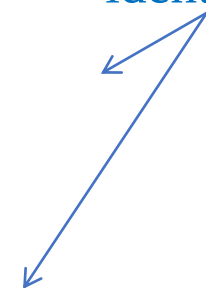
- 衡量了当 x 采样自它的概率分布时，函数 f 的值有多大变化

- 两个函数 $f(x)$ 和 $g(x)$ 的协方差

$$\text{Cov}(f(x), g(y)) = \mathbb{E}[(f(x) - \mathbb{E}[f(x)])(g(y) - \mathbb{E}[g(y)])]$$

- 它衡量了两个值之间的线性关系，以及这些变量变化的大小
- 协方差的绝对值很大意味着这些变量变化很大，并且都远离它们各自的平均值
- 随机向量 $\mathbf{x} \in \mathbb{R}^n$ 的协方差矩阵是一个 $n \times n$ 的矩阵
$$\text{Cov}(\mathbf{x})_{i,j} = \text{Cov}(x_i, x_j)$$

f 和 g 可以是
identity functions



● 常见的概率分布

- 伯努利分布(Bernoulli distribution): 二元随机变量

$$P(x = 1) = \phi, P(x = 0) = 1 - \phi$$

$$P(x = x) = \phi^x (1 - \phi)^{1-x}$$

$$\mathbb{E}_x[x] = \phi, \text{Var}(x) = \phi(1 - \phi)$$

- 范畴分布(Multinoulli/categorical distribution): 具有 k 个不同状态的离散随机变量, 其中 k 是有限的

$$P(x = i | \mathbf{p}) = p_i$$

其中 $\mathbf{p} \in [0,1]^k$ 并且 $\sum_{i=1}^k p_i = 1$

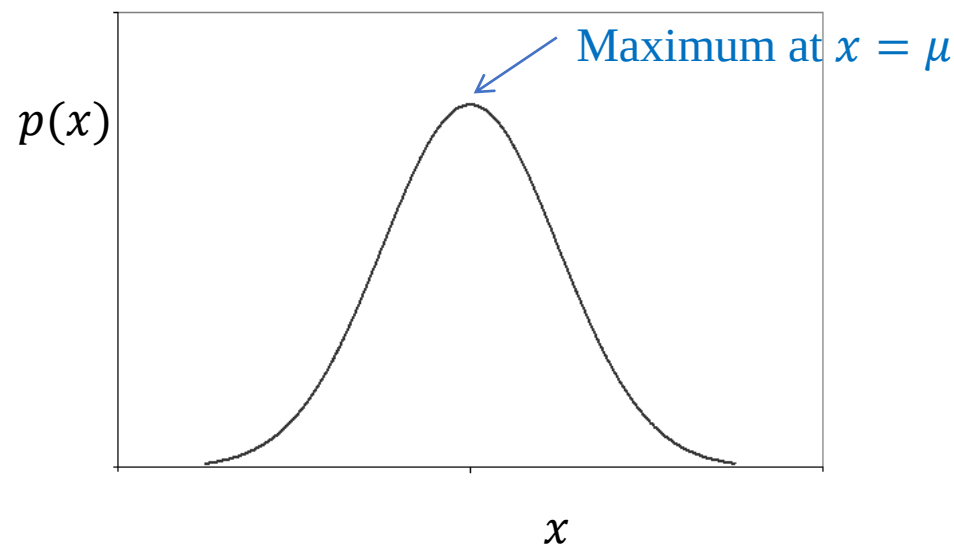
● 常见的概率分布

- 高斯分布或正态分布(Gaussian/normal distribution): 连续的随机变量

$$N(x; \mu, \sigma^2) = \sqrt{\frac{1}{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

- 均值: $\mathbb{E}[x] = \mu$
- 方差: $\text{Var}[x] = \sigma^2$
- 标准差: σ

中心极限定理表明大量独立随机变量之和近似服从正态分布



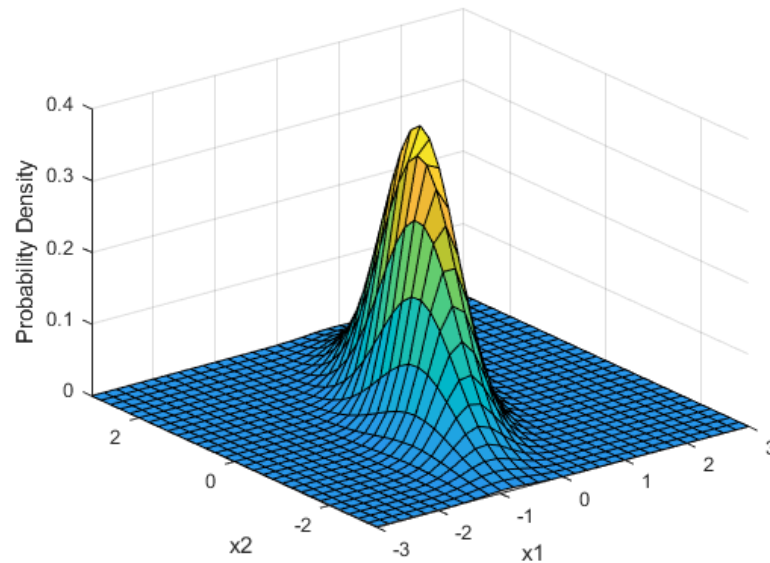
● 常见的概率分布

- 多元正态分布(Multivariate normal distribution): 连续的随机向量

$$N(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sqrt{\frac{1}{(2\pi)^2 \det(\boldsymbol{\Sigma})}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

- 均值: $\mathbb{E}[\mathbf{x}] = \boldsymbol{\mu}$
- 协方差: $\text{Cov}[\mathbf{x}] = \boldsymbol{\Sigma}$

各向同性高斯分布:
 $\boldsymbol{\Sigma} = \alpha \mathbf{I}$ 其中 α 是一个标量



● 贝叶斯定理

x 的后验概率

x 的先验概率

给定 x 后 y 的概率; 似然

y 的先验概率

$$P(x|y) = \frac{P(x)P(y|x)}{P(y)}$$



Thomas Bayes
(1702~1761)

● 数学基础

优化

● 基于梯度的优化

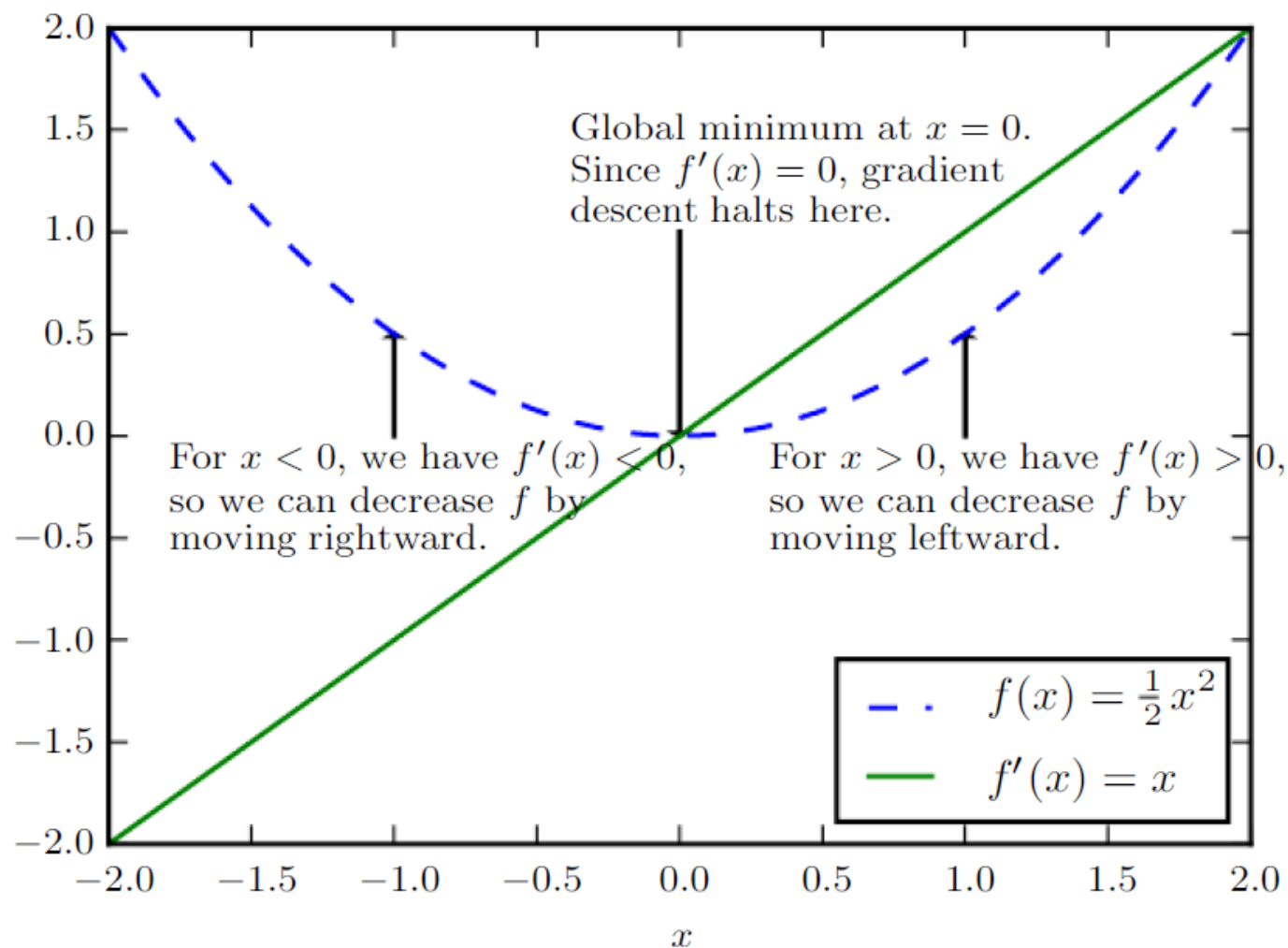
- 我们想最大化或最小化的函数称为目标函数
- 当最小化目标函数时，也可以称其为成本函数，损失函数或误差函数
- 函数 $y = f(x)$ 的导数表示为 $f'(x)$ 或 $\frac{dy}{dx}$ ，含义为 f 在 x 处的斜率或梯度
- 梯度下降

$$x' = x - \eta f'(x)$$

其中 $\eta > 0$ 表示学习率(learning rate)

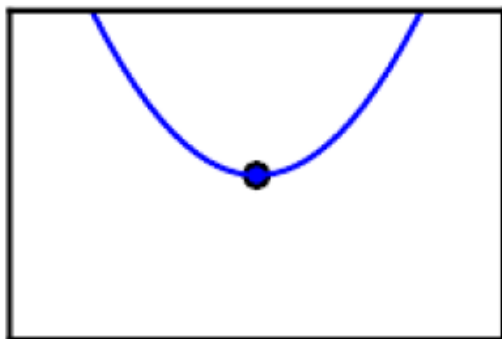
● 梯度下降

$$x' = x - \eta f'(x)$$

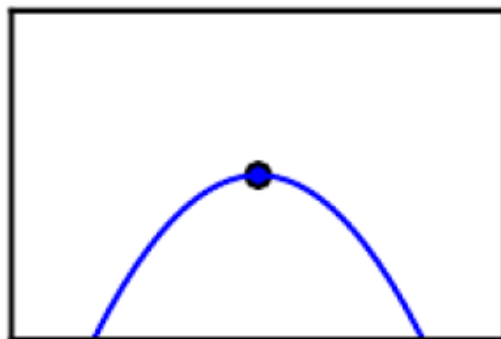


● 临界点

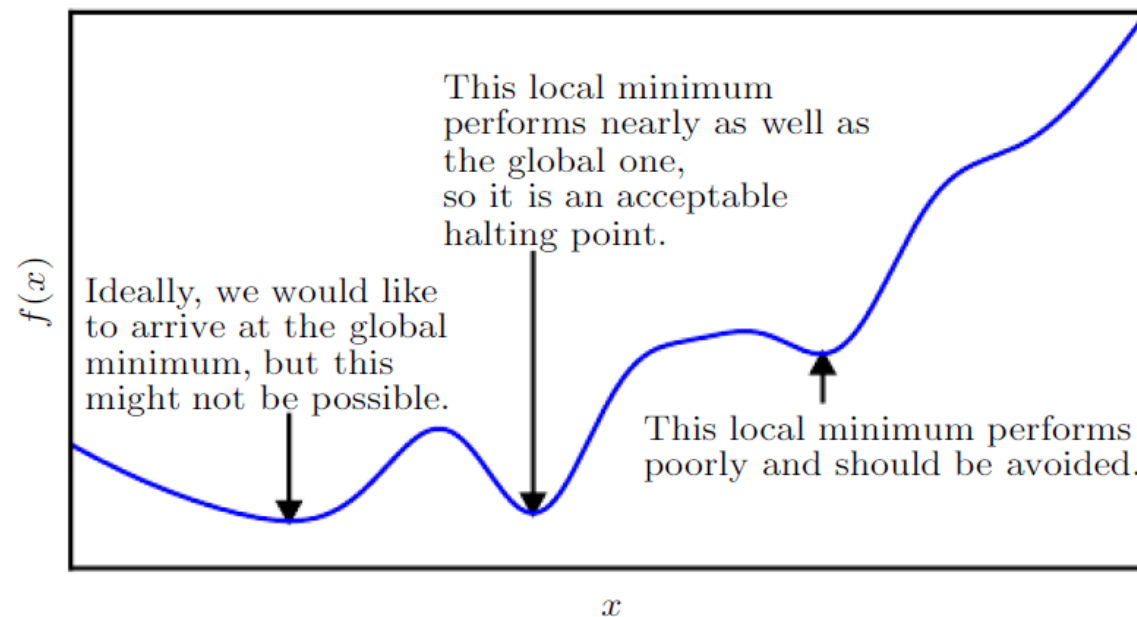
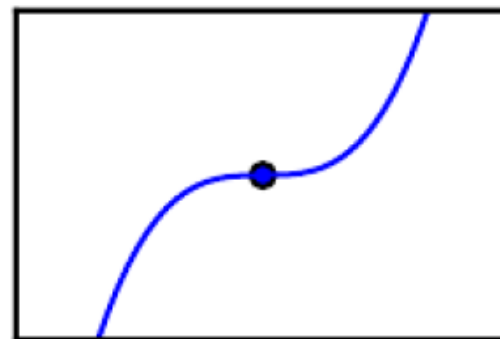
Minimum



Maximum



Saddle point



● 多元函数的梯度下降

- 对于单变量的函数 $y = f(x)$, 梯度下降的方法为
$$x' = x - \eta f'(x)$$
- 对于函数 $y = f(\mathbf{x})$, 偏导数表示为 $\partial f / \partial x_i$
- 梯度下降的方法变成

$$\mathbf{x}' = \mathbf{x} - \eta \nabla_{\mathbf{x}} f(\mathbf{x})$$

其中 $\eta > 0$ 并且 $\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{pmatrix} \partial f / \partial x_1 \\ \partial f / \partial x_2 \\ \dots \\ \partial f / \partial x_n \end{pmatrix}$

● 微积分运算法则

- 链式法则: 复合函数 $f(g(x))$ 的导数
$$[f(g(x))]' = f'(g(x))g'(x)$$

莱布尼茨记法

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

- 乘法法则: 两个函数相乘的导数
$$(f \cdot g)' = f' \cdot g + f \cdot g'$$

莱布尼茨记法

$$\frac{d}{dx}(u \cdot v) = \frac{du}{dx} \cdot v + u \cdot \frac{dv}{dx}$$

- 除法法则

$$\frac{d}{dx} \left(\frac{f(x)}{g(x)} \right) = \frac{f'g - fg'}{g^2}$$

● 两步求导

假定我们有:

- 独立的输入变量, $x_1, x_2, \dots, x_n$
- 相关的中间变量, $u_1, u_2, \dots, u_m$, 每一个中间变量都是 $x_1, x_2, \dots, x_n$ 的函数
- 相关的输出变量, $w_1, w_2, \dots, w_p$, 每一个输出变量都是 $u_1, u_2, \dots, u_m$ 的函数

对于任意的 $i \in \{1, 2, \dots, p\}$ 和 $j \in \{1, 2, \dots, n\}$ 可以推出

$$\frac{\partial w_i}{\partial x_j} = \sum_{k=1}^m \frac{\partial w_i}{\partial u_k} \frac{\partial u_k}{\partial x_j}$$

对中间变量求和

来自 wikipedia

● 小结

- 线性代数

- 数学对象：标量，向量，矩阵，张量
- 简单运算：矩阵转置，求逆，乘法
- 范数： L_p 范数

- 概率论

- 随机变量：离散，连续
- 概率分布：PMF 和 PDF
- 边缘概率分布

- 条件概率
- 独立和条件独立
- 期望，方差和协方差
- 常见的概率分布
- 贝叶斯定理

- 优化

- 梯度下降
- 临界点
- 微积分运算法则

● 大纲

- 数学基础
- 机器学习基础

● 学习算法

“A computer program is said to **learn** from experience E w.r.t. some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .” ---Tom Mitchell, 1997

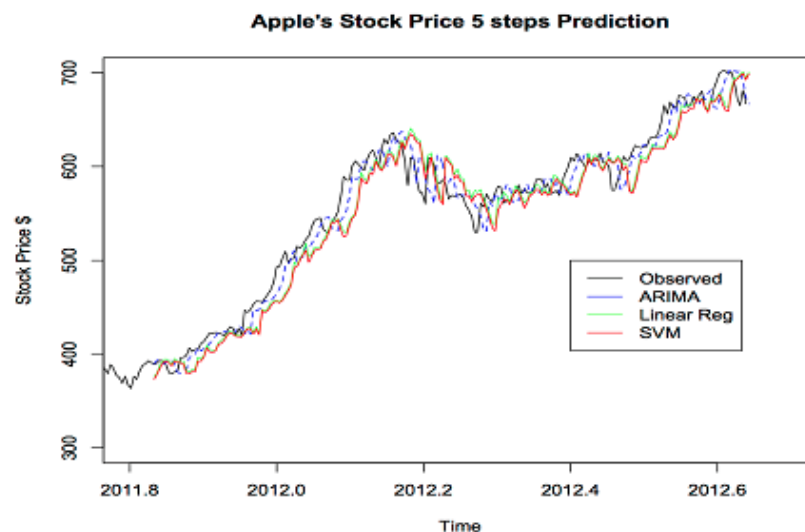
- 机器学习(ML) 的任务通常以ML系统应如何处理**样本(example)**的方式进行描述
- 一个样本是从某些对象或事件中进行定量测量的**特征(features)**的集合
 - 铲斗的特征:颜色、直径、高度、材料等
 - 动物的特征:大小、形状、腿的数量等



● 任务 T

• 分类

- 假定有 k 个类别。寻找函数 $f: \mathbb{R}^n \rightarrow \{1, \dots, k\}$



• 回归

- 寻找函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, 其中 m 通常是 1

回归结果可能会转换为分类结果

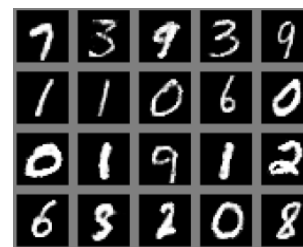
● 任务 T

- 生成和采样

数据集



用GAN生成



- 降噪



● 任务 T

- 转录

输入



输出



Deep learning is cool

- 机器翻译



● 任务 T

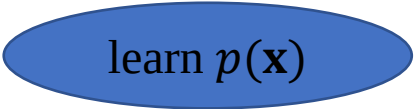
- 结构预测
- 异常检测
- 生成和采样
- 估算缺失值
- 密度估算
- 等等...

● 评价指标 P

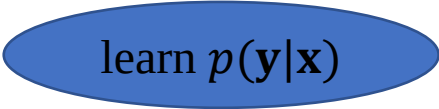
- 评价指标需要可以定量评估机器学习 (ML) 系统的性能
- 评价指标 P 通常取决于系统要执行的**任务 T**
 - 分类和转录: 准确率或错误率
 - 回归和降噪: 预测与真实之间的距离
 - 生成, 机器翻译: 比较困难, 有时需要人类评估
- 评价指标是系统在**测试集(test set)**上的性能度量, 测试集与用于训练系统的训练集是分离的

● 经验 E

- 根据将哪种类型的经验用于训练，可以将包括深度学习算法在内的机器学习算法大致划分为无监督(unsupervised)和有监督(supervised)两类
- 算法经验一般是数据集，它是很多样本或数据点的集合，样本和数据点一般用 x 表示
 - 我们可以将样本视为随机变量 x 的采样
- 无监督学习算法
 - 数据集包含很多特征，无监督学习算法是学习数据集中有用的属性或结构
- 有监督学习算法
 - 数据集包含很多特征，并且每一个样本都有标签或目标，用 y 表示



learn $p(x)$



learn $p(y|x)$

● 示例: 线性回归

- 任务 T : 用 x 预测 y , $\hat{y} = \mathbf{w}^\top \mathbf{x}$
- 评价指标 P : 模型在包含 m 个测试样本 $\{(\mathbf{x}_i, y_i)\}^{\text{test}}$ 的测试集上的均方误差

x_i : 特征
 w_i : 权重

$$\text{MSE}_{\text{test}} = \frac{1}{m} \sum_i (\hat{y}_i - y)^{\text{test}}$$

- 经验 E : 最小化模型在包含 q 个训练样本 $\{(\mathbf{x}_i, y_i)\}^{\text{train}}$ 的训练集上的均方误差

$$\text{MSE}_{\text{train}} = \frac{1}{q} \sum_i (\hat{y}_i - y)^{\text{train}}$$

- 令 $\{(\mathbf{x}_i, y_i)\}^{\text{train}}$ 整体表示为 $(\mathbf{X}^{\text{train}}, \mathbf{y}^{\text{train}})$, 则

$$\begin{aligned} \nabla_{\mathbf{w}} \text{MSE}_{\text{train}} &= \nabla_{\mathbf{w}} \frac{1}{q} \left\| \hat{\mathbf{y}}^{\text{train}} - \mathbf{y}^{\text{train}} \right\|_2^2 = 0 \\ \Rightarrow \mathbf{w} &= \left(\mathbf{X}^{\text{train}^\top} \mathbf{X}^{\text{train}} \right)^{-1} \mathbf{X}^{\text{train}^\top} \mathbf{y}^{\text{train}} \end{aligned}$$

● 容量，过拟合和欠拟合

- ML算法需要在新的、没有训练过的输入上表现良好，而不是仅仅限于训练数据
 - 这种能力被称为泛化能力(generalization)



- 更小的训练误差 → 更高的模型容量
 - 如果训练误差非常大，则表明模型在训练集上欠拟合
- 更小的测试误差或泛化误差 → 更高的泛化能力
 - 如果训练误差很小但测试误差非常大，则表明模型在训练集上过拟合

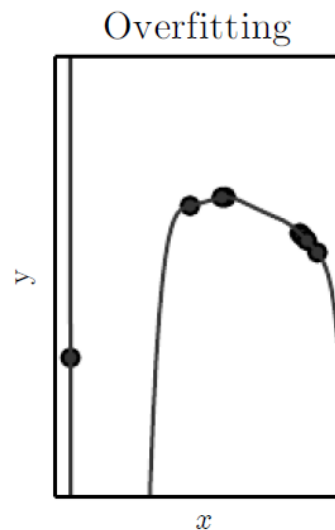
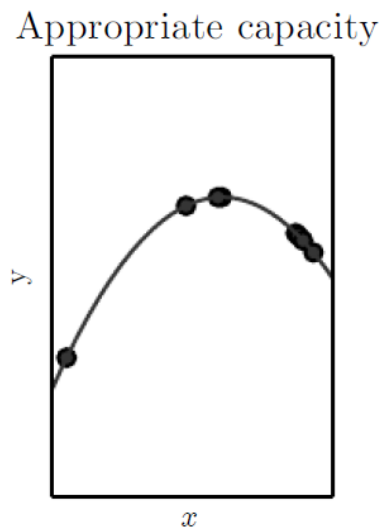
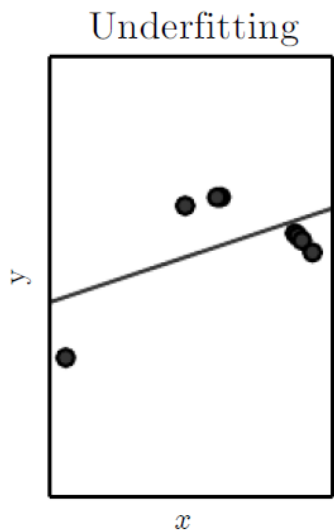
● 示例: 多项式回归

- 考虑一个回归问题, 输入 x 和 输出 y 都是标量。寻找一个函数 $f: \mathbb{R} \rightarrow \mathbb{R}$ 来拟合数据

- $f(x) = b + wx$
- $f(x) = b + w_1x + w_2x^2$
- $f(x) = b + \sum_{i=1}^9 w_ix^i$

MSE 训练:

$$\min_w \frac{1}{N} \sum_{n=1}^N \|f(x^{(n)}) - y^{(n)}\|_2^2$$



● 一般原则

- 增大模型容量
 - 减小训练误差
- 增强泛化能力
 - 缩小训练误差和测试误差之间的差距

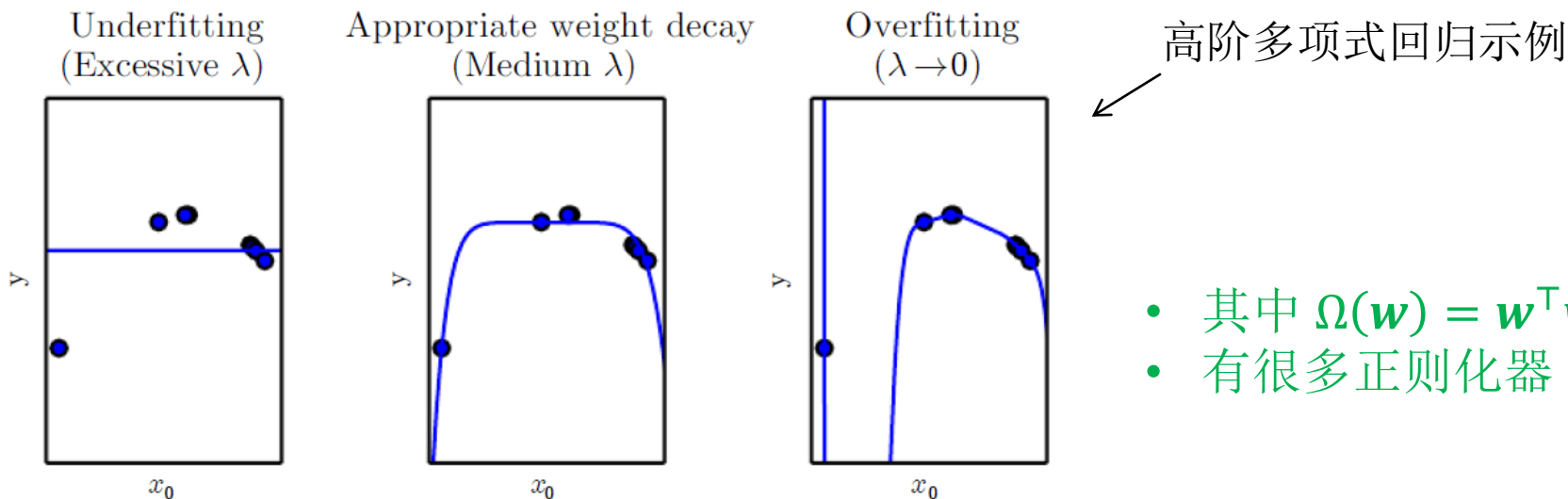
正则化

- 为执行特定的任务，我们经常在学习算法中设置一些偏好(preferences)，这些偏好通过正则化(regularizer) Ω 来体现
- 例如，对于多项式回归，成本函数为

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$$

其中 $\lambda > 0$ 是一个常数.

权重衰减



● 正则化

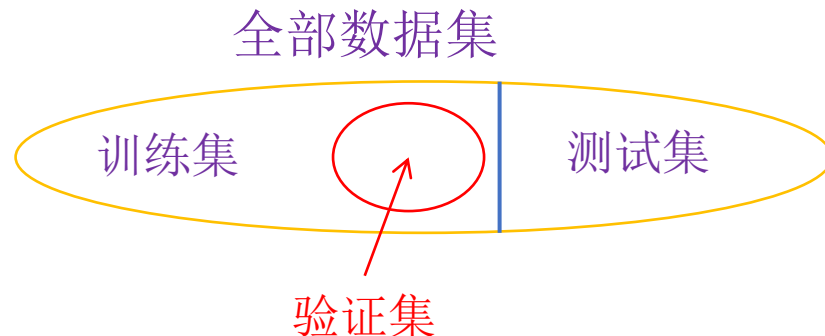
正则化是我们旨在减少模型泛化误差而不是训练误差，从而对学习算法所做的任何修改

● 超参数

- 许多机器学习算法都有两组参数:
 - 超参数(Hyperparameters): 控制算法的表现, 不会被算法更新. 超参数通常决定了模型的容量
 - 可学习参数 (“可学习” 通常忽略): 可以从数据中学习到的参数
- 多项式回归算法 $J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$
 - 超参数: λ
 - 可学习参数: $\mathbf{w}$
- 神经网络
 - 超参数: 网络层数, 每层的神经元个数, 等等...
 - 可学习参数: 每层的权重和偏置

● 验证集

- 如果我们无法获得测试集，该如何选择超参数？
 - 是否可以设置让训练误差尽可能小的超参数？
- 我们需要一个模型没有在其上训练过的数据集合
 - 尽可能减小模型在这个数据集合上的误差
 - 这个数据集合被称为**验证集(validation set)**
- 应该如何获取验证集？



● 最大似然估计

问题定义

- 给定 N 个样本 $\mathbb{X} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}\}$ 的数据集合，每个样本独立采样于未知的真实数据分布 $p_{\text{data}}(\mathbf{x})$
- 寻找概率分布 $p_{\text{model}}(\mathbf{x}; \boldsymbol{\theta})$ 来近似 $p_{\text{data}}(\mathbf{x})$
- 任务目标是找出最优的 $\boldsymbol{\theta}$

- 对参数 $\boldsymbol{\theta}$ 的最大似然估计定义为

$$\boldsymbol{\theta}_{\text{ML}} = \arg \max_{\boldsymbol{\theta}} p_{\text{model}}(\mathbb{X}; \boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^N p_{\text{model}}(\mathbf{x}^{(i)}; \boldsymbol{\theta})$$

- 通常用

$$\boldsymbol{\theta}_{\text{ML}} = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^N \log p_{\text{model}}(\mathbf{x}^{(i)}; \boldsymbol{\theta})$$

对数似然

$$= \arg \max_{\boldsymbol{\theta}} \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(\mathbf{x}; \boldsymbol{\theta})$$

- 其中 $\hat{p}_{\text{data}}$ 是经验分布

● 条件对数似然

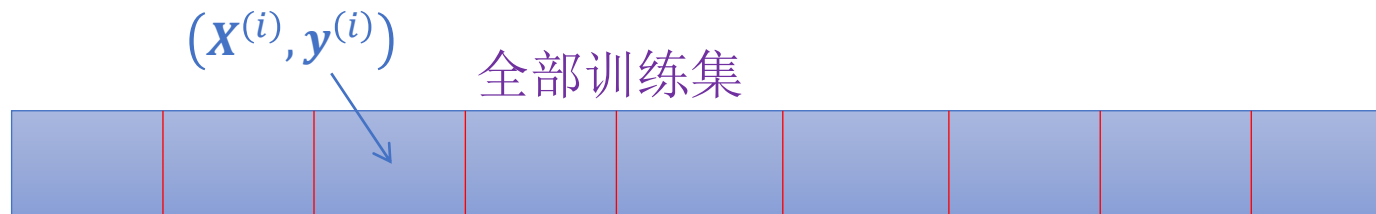
- 估计条件概率 $P(y|x; \theta)$ 由此来给定 x 预测 y
 - 例如, 对于分类任务, y 是一个 (离散) 随机变量, 代表输入 x 的标签
- 如果 X 代表所有输入, Y 代表所有观测的目标, 则条件最大似然估计 为

$$\theta_{\text{ML}} = \arg \max_{\theta} P_{\text{model}}(Y|X; \theta)$$

- 如果所有的样本都是独立同分布的, 那么上式可以分解为

$$\theta_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^N \log P_{\text{model}}(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}; \theta)$$

● 随机梯度下降



- 在整个训练集中最小化成本函数的计算开销非常大
- 我们通常将训练集划分为较小的子集或 **minibatches** 然后在单个minibatches $(X^{(i)}, y^{(i)})$ 上优化成本函数，并取平均值

$$J(\theta) = \frac{1}{N'} \sum_{i=1}^{N'} L(X^{(i)}, y^{(i)}, \theta)$$

$$g = \frac{1}{N'} \nabla_{\theta} \sum_{i=1}^{N'} L(X^{(i)}, y^{(i)}, \theta)$$
$$\theta = \theta - \eta g$$

- 共 N' 个 minibatches
- minibatches 的大小可以取1到几百

● 总结

- 数学基础
 - 线性代数
 - 概率论
 - 优化
- 机器学习基础
 - 学习算法：任务，评价指标，经验
 - 容量，过拟合和欠拟合
 - 超参数和验证集
 - 最大似然估计
 - 随机梯度下降

● 参考书目

- Chapters 2-5 in Deep Learning by Goodfellow, Bengio and Courville, 2016, MIT Press